CS434: Machine Learning and Data Mining

Homework 4: Neural Networks

Chase Mackenzie | mackench@oregonstate.edu

Kaggle Team: Chase Mackenzie

1. Implementing a Neural Network for Digit Identification

1.1 Q1: Backward Pass for a Linear Layer

The backward pass needs to compute dL/dW, dL/db, and dL/dX given the upstream gradient dL/dZ.

For dL/dW, each column i of the gradient equals the input x scaled by the loss gradient of z_i. Summing over a batch of n examples gives dL/dW = X^T(dL/dZ), which works out to shape (input_dim x output_dim).

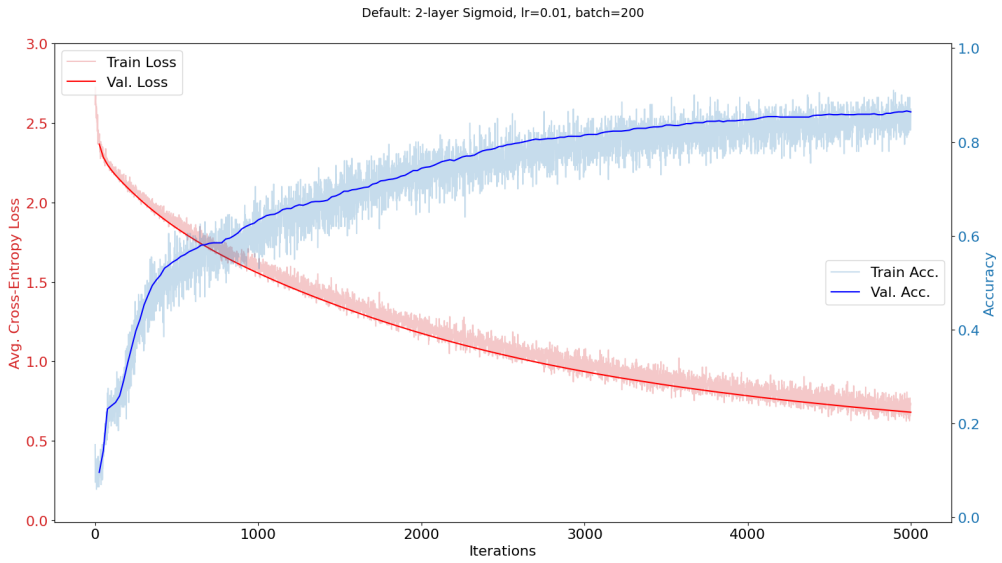
For dL/db, since each b_i only affects z_i the Jacobian is the identity, so the gradient is just the sum of the rows of dL/dZ, giving shape (1 x output_dim).

For dL/dX, taking the derivative of z_i with respect to x gives W^T, so dL/dX = (dL/dZ)W^T, shape (n x input_dim).

The implementation is straightforward:

```
self.grad_weights = self.input.T @ grad
self.grad_bias = np.sum(grad, axis=0, keepdims=True)
return grad @ self.weights.T
```

Running the default configuration (2-layer Sigmoid, lr=0.01, batch size 200, 200 epochs) produces the following learning curve:

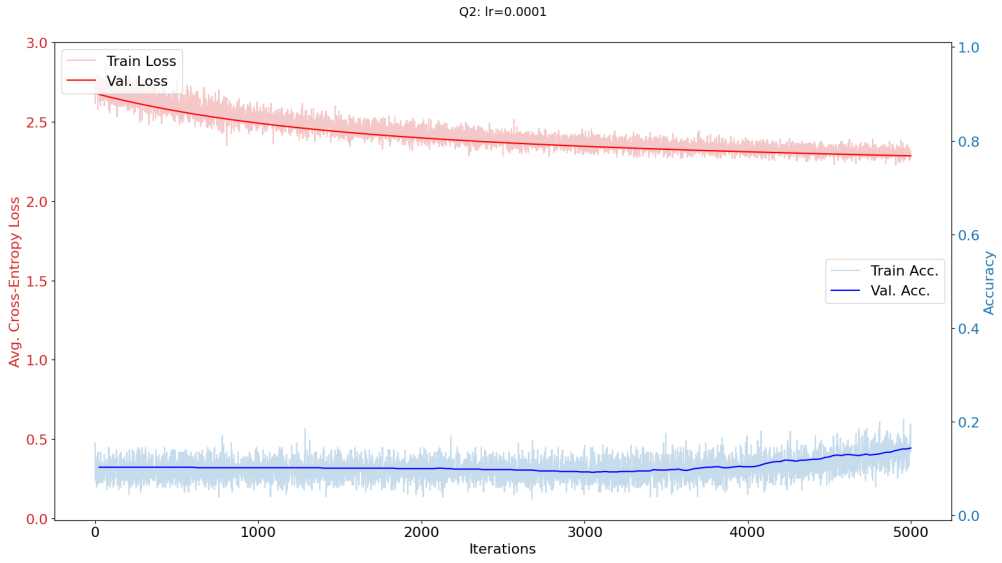


The model reaches 86.4% validation accuracy. Loss decreases smoothly over training and the train/val curves stay close together, which suggests the model is not overfitting much at this size.

1.2 Q2: Learning Rate

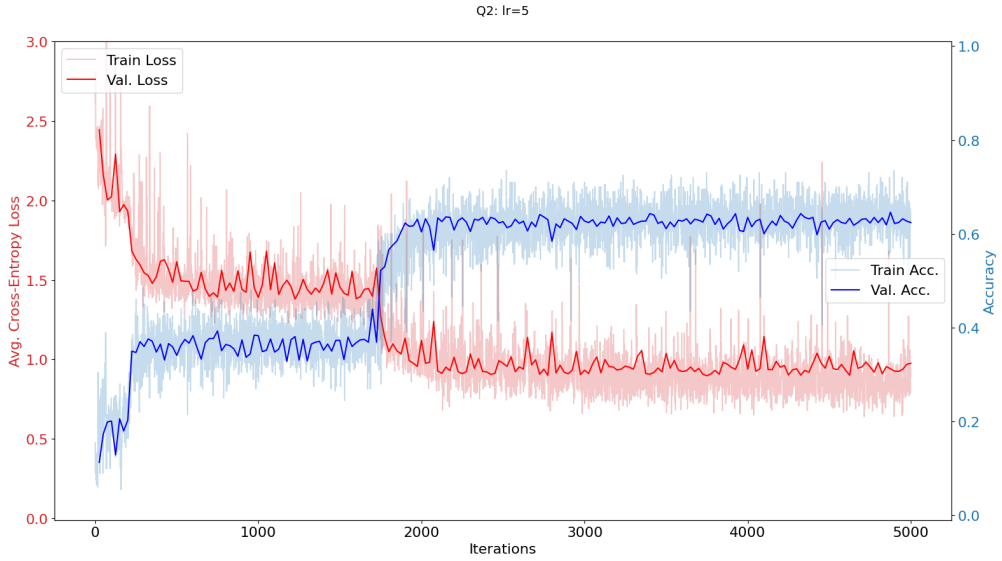
I tested three step sizes while leaving everything else at the defaults.

lr = 0.0001 — final val acc: 14.4%



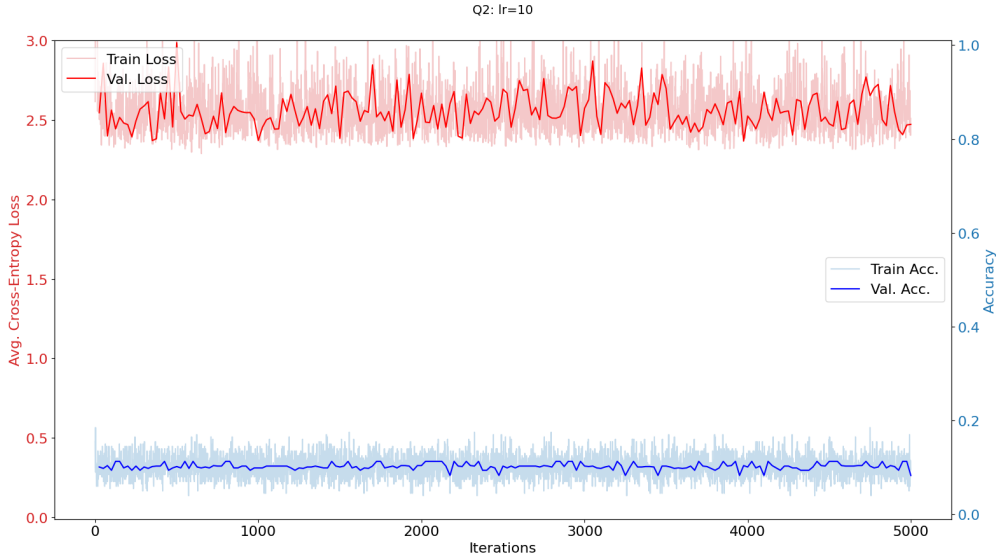
The curve is very smooth but almost flat. The model barely moves because the steps are so small — after 200 epochs it has gone from ~2.7 loss down to ~2.3, which is still close to random. It looks like it is learning but the accuracy tells a different story.

lr = 5 — final val acc: 62.4%



Much noisier. The loss bounces around heavily each iteration because the steps keep overshooting. The model does pick up some signal and gets to about 62%, but it never really settles — the accuracy oscillates throughout training rather than converging.

lr = 10 — final val acc: 8.3%



This one basically fails. The updates are so large the weights get blown around and the model never learns, sitting near random chance the whole time.

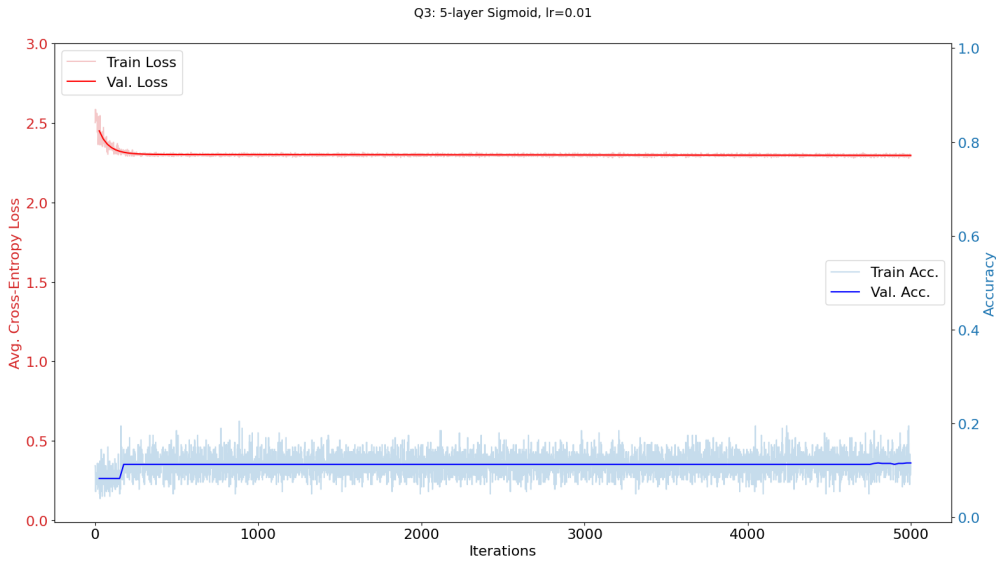
(a) Compared to the default, lr=0.0001 is smooth but makes no real progress; lr=5 is noisy and plateaus well below default; lr=10 diverges. The default lr=0.01 finds the right balance between

making progress each step and not overshooting. As learning rate increases the curves get rougher and accuracy drops.

(b) With more epochs, lr=0.0001 would keep improving since the loss was still going down at epoch 200. It would eventually get somewhere useful but would need roughly 100x as many epochs to match the default, since the steps are 100x smaller.

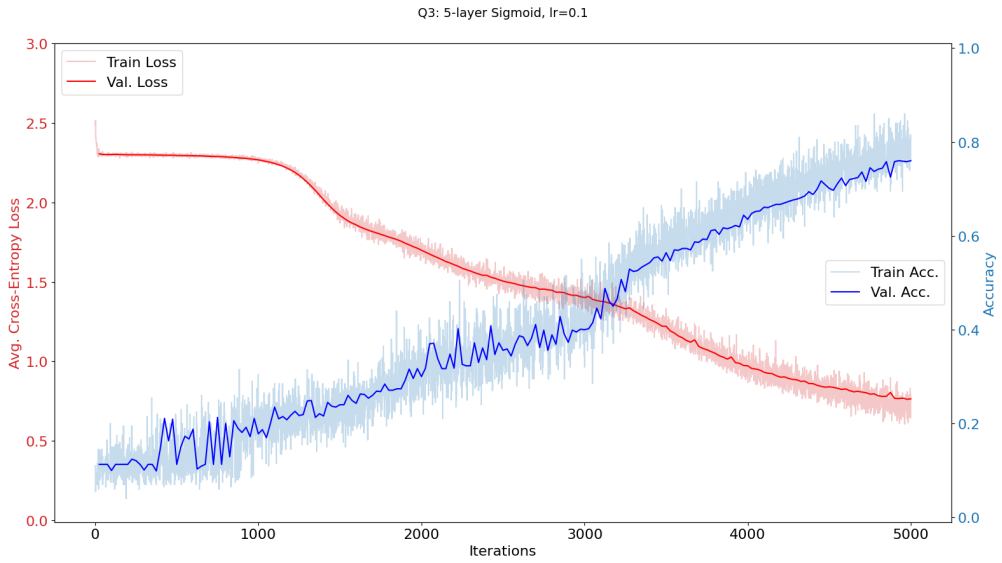
1.3 Q3: ReLU and Vanishing Gradients

5-layer Sigmoid, lr=0.01 — final val acc: 11.6%



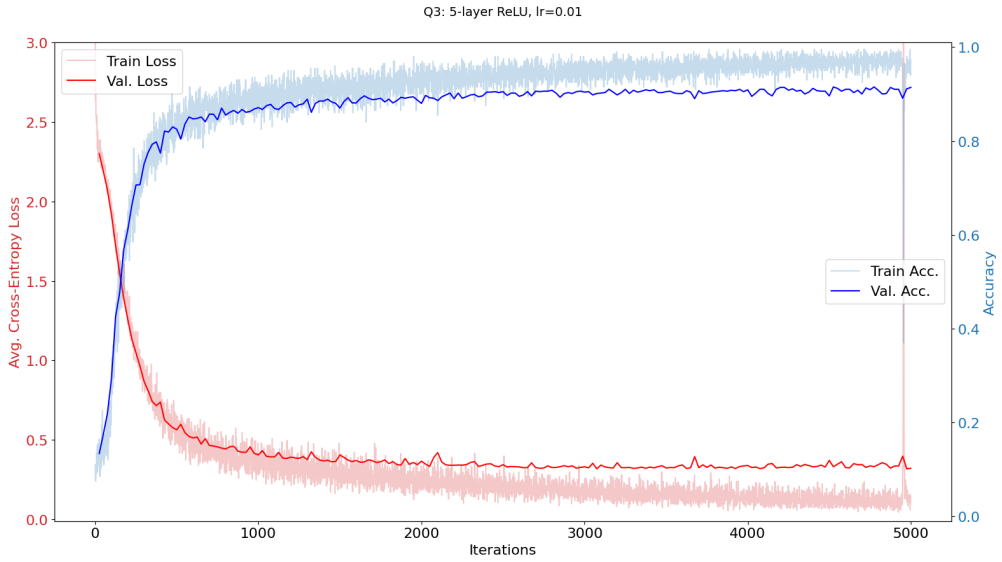
Completely flat. The model does not learn at all with these settings. The gradient has to multiply through five Sigmoid derivatives on the way back to the first layer, and since those derivatives max out at 0.25, by the time the gradient reaches the early layers it is essentially zero. The weights near the input never update.

5-layer Sigmoid, lr=0.1 — final val acc: 76.0%



Increasing the learning rate 10x gets the model learning again. It is still slower than the 2-layer default and the train/val gap is a bit larger toward the end, but it converges to a reasonable result. The higher lr compensates for the tiny gradients reaching the lower layers.

5-layer ReLU, lr=0.01 — final val acc: 91.4%



This one works really well. It learns fast — already above 80% by epoch 20 — and ends up noticeably better than the default. There is a growing gap between train and val by the end (train ~97%, val ~91%) which makes sense since the model has a lot more capacity. ReLU fixes the vanishing gradient problem that killed the Sigmoid version.

(a) The 5-layer Sigmoid at lr=0.01 is far worse than the 2-layer default (11.6% vs 86.4%) — adding depth without fixing vanishing gradients just makes things worse. Bumping lr to 0.1 helps substantially but still trails the default. The ReLU version at the same lr=0.01 beats everything, and it is also the one that shows the most train/val divergence, meaning it is a higher-capacity model that could overfit if not careful.

(b) The higher learning rate helps with the 5-layer Sigmoid because the gradient arriving at lower layers is tiny due to repeated multiplication by small Sigmoid derivatives. A larger step size scales those small gradients back up enough to produce visible weight changes. It does not actually fix vanishing gradients structurally, but makes them workable in practice.

(c) ReLU's derivative is either 0 or 1 — when a neuron is active, the gradient passes through that layer completely unchanged. Sigmoid squashes everything to (0,1) with a derivative that peaks at 0.25, so five layers of that multiplies the gradient by at most 0.25^5 which is around 0.001. ReLU does not have this problem, so gradients reach the early layers intact and training works normally.

1.4 Q4: Measuring Randomness

I ran the default configuration five times with different random seeds:

Seed	Final Val Accuracy
0	86.1%
7	87.8%
42	88.8%
99	88.4%
2024	89.4%

The results span about 86% to 89%, so randomness alone can swing accuracy by roughly 3 points. This matters for the previous questions — differences of 1-2% between configurations could just be noise. The big effects we saw (like ReLU at 91% vs Sigmoid at 11%) are clearly real, but smaller differences should be taken with some skepticism without averaging over seeds.

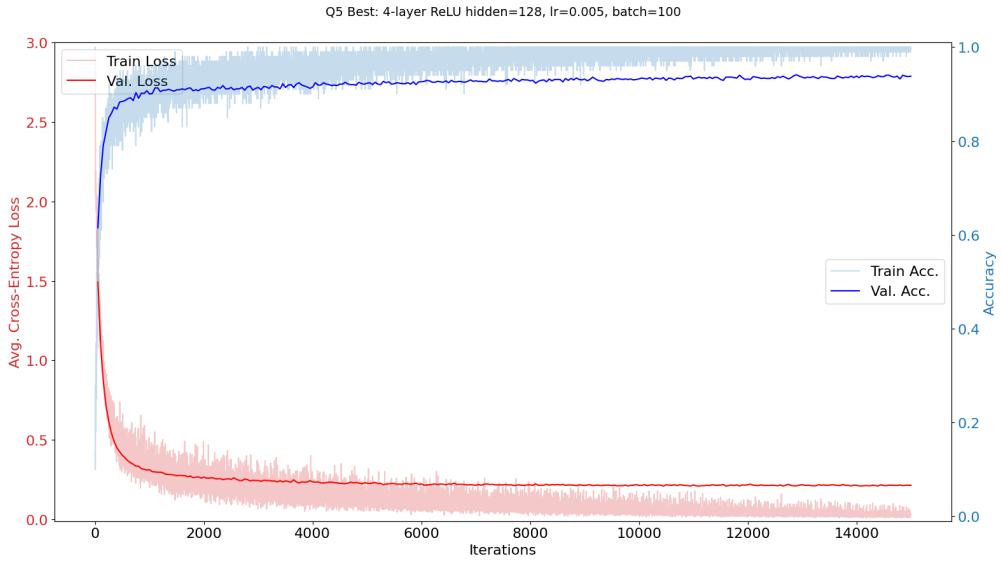
1.5 Q5: Kaggle Submission

For the final submission I made the following changes from the default:

Architecture: 4 layers with hidden dimensions 256, 128, and 64 (vs the default 1 hidden layer of size 16). Activation: ReLU throughout. Learning rate: cosine decay from 0.01 down to 0.0001 over 400 epochs. Batch size: 256. Weight decay: 1e-4. Training data: combined train and validation sets (all

6,000 labeled examples).

The main motivation was the Q3 results — ReLU with depth works much better than Sigmoid, so I scaled that up. Training on the full labeled dataset rather than holding out a validation split is standard when submitting final predictions since there is no longer a reason to hold data back.



The model trained only on the original training set reached 93.8% val accuracy. After retraining on all 6,000 examples, the val accuracy (measuring against the held-out val set, which the model now trained on) reached 98.7%, which is expected — it has seen that data. The Kaggle score will reflect test set performance.